Python Full Stack Development — Training Material

> **One material policy for the Python program.** Python/Django notes live in this folder.
> **Shared topics** (also for Java FS): `../FRONTEND/` (HTML/CSS/JS + `REACT/`), `../MYSQL/README.md`.
> Pointers only: `06_Frontend/`, `10_React/`. PDFs: `PDF/PYTHON/`.

Source syllabus: KSR Datavizon *FULL STACK PYTHON 2026–27*
Audience: Learners preparing for entry-level Python / Django / full-stack roles
Default schedule: 20 weeks × 1 hour 15 minutes/day (Mon–Fri live) — see Session Calendar
Live contact = 125 hours (100 sessions) + daily homework
Stack: Python 3 · MySQL 8+ · Django 5.x · HTML/CSS/JS · React · Flask (overview) · Git/GitHub · VS Code · Postman

---

Start here

1. **Session Calendar** — 100 sessions, one focus each
2. **Tested Toolchain** · **Project deliverables** · **Capstone one-pager**
3. **Lab Assets** · **Biweekly Assessments**
4. Course Contents (syllabus map)
5. Open the **same** module files in Session Calendar order
6. Optional: Professional Python Deep Dive

Typical class shape (75 min)

Minutes	Block
5	Open lab kit
25	Concept demo / notes
35	Lab
10	Exit + homework

Teaching order

Schedule authority: always `Session_Calendar_100.md` (order below matches it).

Order	Path	Topic
1	01_Foundations/01_Computer_and_Programming_Foundations.md	Computer & programming foundations
2	02_Core_Python/01_Python_Setup_and_First_Programs.md	Python setup & first programs
3	02_Core_Python/02_Variables_Datatypes_Operators_IO.md	Variables, types, operators, I/O

Order	Path	Topic
4	02_Core_Python/03_Flow_Control.md	Flow control
5	02_Core_Python/04_Strings.md	Strings
6	02_Core_Python/05_Lists.md	Lists
7	02_Core_Python/06_Tuples_and_Sets.md	Tuples & sets
8	02_Core_Python/07_Dictionaries.md	Dictionaries
9	02_Core_Python/08_Data_Structures_Overview.md	Data structures overview
10	03_Advanced_Python/01_Functions_Decorators_Generators.md	Functions, decorators, generators
11	03_Advanced_Python/02_Modules_and_Packages.md	Modules & packages
12	03_Advanced_Python/03_OOP.md	OOP (index) → Part 1 · Part 2 · Part 3 · Part 4
13	03_Advanced_Python/04_Exception_Handling.md	Exception handling
14	03_Advanced_Python/05_File_Handling_and_Logging.md	Files & logging
15	04_Git_GitHub/01_Git_and_GitHub.md	Git & GitHub
16	05_Database_MySQL/01_MySQL_for_Full_Stack_Developers.md	MySQL bridge + required labs — map: `MYSQL/README.md`
17	03_Advanced_Python/06_Database_Programming_Bridge.md	Python ↔ MySQL
18	../FRONTEND/01_HTML.md	HTML *(shared)*
19	../FRONTEND/02_CSS_and_Responsive.md	CSS & responsive *(shared)*
20	../FRONTEND/03_JavaScript_DOM_Forms.md	JavaScript, DOM, forms *(shared)*
21	07_Django/01_Django_Introduction_and_Setup.md	Django setup
22	07_Django/02_URLs_Views_Templates_Static.md	URLs, views, templates, static
23	07_Django/03_Models_ORM_Admin.md	Models, ORM, admin
24	07_Django/04_Forms_Sessions_Auth.md	Forms, sessions, auth
25	07_Django/05_Security_and_Deployment.md	Security & deployment (`load_dotenv`, `STORAGES`, `check --deploy`)

Order	Path	Topic
26	07_Django/06_Lab_CRUD_Auth_Consolidation.md	**Protected lab:** CRUD + auth
27	07_Django/07_Lab_Deploy_Capstone_Debug.md	**Protected lab:** deploy + debug
28	08_Flask/01_Flask_Overview.md	Flask overview
29	09_APIs_REST/01_APIs_JSON_Postman_DRF.md	APIs, JSON, Postman, DRF (**full CRUD code in-module**)
30	../FRONTEND/REACT/01_React_Fundamentals.md	React fundamentals *(shared)*
31	../FRONTEND/REACT/02_React_Hooks_Routing_API_Deploy.md	Hooks, routing, API, deploy *(shared)*
32	11_Testing_Clean_Code/01_Testing_Debugging_Clean_Code.md	Testing, debugging, clean code
33	13_Placement/01_Career_Resume_Interview.md	Resume, interview, placement
34	12_AI_Integration/01_AI_Integration_Lite.md	AI lite — **optional stretch only**

Optional enrichment (not numbered above): Professional Python Deep Dive — after Git / Saturday; not on the critical path to Django.

What each module contains

Every teaching module follows the same shape:

- **Module Outcome** (3–5 can-do bullets) right after the header
- **Definition** / **Purpose** before syntax
- Important concepts with **Try these (2 programs)** + runnable code (Core / Advanced Python; expanding elsewhere)
- Step-by-step annotated code
- Practice exercises
- **Mini-Project (Assignment)** — take-home homework with lab kit, Stretch, and **Done when** (when present)
- Interview Q&A
- Graded checks: `16_Assessments/` (biweekly) — not an Assessment block in every module

SQL depth lives in repo `MYSQL/` (required labs listed in the MySQL bridge module) — not duplicated here.

> Mermaid diagrams render on GitHub, many Markdown previewers, and Cursor. PDF builds pre-render them to SVG (`./scripts/build.sh python` needs Node/ `npx` + Chrome). In plain text viewers you still see the diagram source as a readable sketch.

Sequence improvements vs brochure

Brochure order	Improved order (this material)	Why
Django before frontend	HTML/CSS/JS **before** Django	Templates and forms make sense after markup/JS
Git not explicit	Git early (Week 7, then continuous)	Portfolio habit for hiring
React after Django	Kept after Django + APIs	Learners need a backend API to integrate
AI Integration last	**Stretch after** deployed MVP	Never delays core competence / placement
Thin Django calendar	5 teach modules + **2 lab days** (`06`–`07`)	Career-switcher consolidation time

Brochure modules **preserved:** Core Python, Advanced Python (incl. decorators/generators, logging, DB programming), MySQL DDL/DML/DQL/TCL/JOINS, Django full path, Flask, Frontend, React, AI Integration, Agile/resume/live project.

Maintainers

PDFs / optional share mirror (from **repo root**):

```
./scripts/build.sh python         # → PDF/PYTHON/  (same notes as class)
./scripts/pack_student_share.sh   # optional zip mirror; excludes private folders only
```

Share habit: send each module PDF after that module is taught. Rebuilt PDFs under `PDF/` are committed and pushed to origin with the notes.
Private ops (keys, rubrics): `14_Trainer_Only/`.